BhashaCut

Hinglish Video Script Assistant powered by Apertus

Track: Track 2B — Own Project | **Event:** Online | **Team:** Ratnesh Saxena (solo)

Git repo: https://github.com/rsaxena3011/bhashacut

1. Summary

Indian video creators constantly juggle Hindi, English, and Hinglish in scripts, voiceovers, and on-screen text. Most AI models mangle code-switching, producing unnatural output that needs manual fixing. BhashaCut uses Apertus — trained on 1800+ languages with 40% non-English tokens — to polish Hinglish scripts into natural, voiceover-ready text. It also includes an offline TTS hazard detector that flags characters and patterns that break TTS engines (the *kartik* conjunct and ellipsis truncation issues), plus a consistency checker for character names and terminology. The TTS hazard detector works fully offline with no API needed; polishing and consistency features use the Apertus API.

2. Architecture

BhashaCut is a Python CLI tool with three components:

- **Script Polisher** — Sends rough Hinglish to Apertus via OpenAI-compatible API, returns polished natural text in the target language (Hinglish, Hindi, or English).
- **TTS Hazard Detector** — Offline regex-based scanner flagging: *kartik*-style conjuncts (reph + consonant) that crash some TTS backends; ellipsis characters that truncate synthesis; mixed Latin/Devanagari within single words.
- **Consistency Checker** — Tracks character names and terminology across the script, flags spelling variations.

Data flow: input text file → TTS hazard scan (offline) → Apertus polish (API) → consistency check → polished script + hazard report.

Target architecture (mandatory)

a) On-premise — BhashaCut is deployable on-premise. The TTS hazard detector runs fully offline with zero external dependencies. Apertus can be self-hosted via Ollama (*swiss-ai/apertus-8b-instruct*) or vLLM. The Docker container includes all dependencies. At runtime the only external dependency is the configurable Apertus API endpoint; with local Ollama there are zero external network calls.

3. Use of Apertus

Model: *swiss-ai/apertus-8b-instruct* (via Public AI hosted endpoint)
How it is used: inference
Where it runs: hosted endpoint https://api.publicai.co/v1 (also supports local Ollama)

The polisher sends the raw script with a system prompt instructing Apertus to produce natural, voiceover-ready text while preserving meaning, tone, and code-switching patterns appropriate to the target language. Temperature 0.3 for consistent output. The model choice leverages Apertus's multilingual training (1800+ languages, 40% non-English tokens), which handles Hindi-English code-switching better than English-centric models.

Environment variables: *LLM_NAME* (default *swiss-ai/apertus-8b-instruct*), *LLM_BASE_URL* (default https://api.publicai.co/v1), *LLM_API_KEY*.

4. Data

No training data. The tool processes user-provided scripts at runtime. Test scripts in *data/* are original Hinglish samples written for this project, released under CC-BY-4.0. No personal data is included.

5. Evaluation

Task: polish rough Hinglish video scripts and detect TTS hazards.

Setup	Metric	Result
TTS Hazard Detector	Hazard recall on 3 known patterns	3/3 detected (100%)
Apertus Polishing	Manual review: naturalness	Pending API access
Baseline (no tool)	Manual editing time	N/A (qualitative)

The TTS hazard detector was tested against three known failure patterns from production experience and correctly flagged all three. Full Apertus-backed evaluation awaits API access.

6. Limitations

- Apertus-backed polishing and consistency features require an API key and network access. Without a key, only the offline TTS hazard detector works.
- The hazard detector uses regex patterns; it may miss novel TTS failure modes not in its pattern list.
- Consistency checking is basic string matching; semantic equivalence (e.g. 'Ram' vs 'Rama') requires manual alias definition.
- Tested only on short scripts (< 5000 words). Long-form handling not yet validated.

7. Reproducibility

Hardware: any x86_64 machine with Docker. No GPU required (API-based inference).

```
cd track_2b && make run
```

This builds the Docker image and runs the TTS hazard detector on the sample script. For full Apertus features, set *LLM_API_KEY* before running. See git log for the exact submission commit.

8. Next steps

With another month: (1) web UI for non-technical creators, (2) fine-tune hazard patterns from real TTS engine feedback, (3) batch processing for multi-episode series, (4) video-editing timeline plugin API, (5) more Indian language pairs (Tamil-English, Bengali-English).

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Apertus model: https://huggingface.co/swiss-ai/Apertus-v1.5-8B
- Public AI Inference Utility: https://publicai.co
- Public AI developer docs: https://platform.publicai.co